Compte rendu – Site web des Jeux Olympiques de Los Angeles 2028

Épreuve E6 · BTS SIO option SLAM · Lycée La Tournelle

KANSIETOKO Yuung Addi

Octobre – novembre 2026 · PHP / MySQL

1. Contexte et objectif

Ce projet est mon projet de fin d'études, présenté à l'épreuve E6 du BTS SIO option SLAM. Le comité d'organisation des Jeux Olympiques de Los Angeles 2028 a besoin d'un site web avec deux parties : une partie publique qui présente les sports, le calendrier des épreuves et les résultats, et un back-office réservé aux administrateurs, pour gérer toutes ces données.

Le professeur nous a fourni un code de départ en PHP et deux scripts SQL. J'ai travaillé comme un développeur qui reprend un existant : j'ai d'abord audité et corrigé ce code, puis j'ai développé les fonctionnalités manquantes, je les ai testées et je les ai mises en ligne. Tout est en PHP natif, sans framework, pour maîtriser chaque mécanisme et pouvoir l'expliquer.

Élément	Valeur
Langages	PHP 8.3 (PDO), SQL (MySQL), HTML5, CSS3
Environnement local	MAMP (Apache, MySQL 5.7), VS Code, phpMyAdmin
Tests	PHPUnit 11 installé avec Composer
Gestion de projet	Trello (méthode Kanban), Git et GitHub
Site en ligne	https://yuung404.infinityfree.me (InfinityFree, HTTPS)
Code source	https://github.com/Yuung404/jo2028-kansietoko-yuung

2. Organisation du projet

J'ai découpé le travail en 8 missions et je l'ai suivi sur un tableau Trello à trois colonnes : À faire, En cours, Terminé. Chaque carte a une échéance et une checklist. La mission 6 (le back-office) est découpée en une carte par table gérée, avec à chaque fois la même checklist : liste, ajout, modification, suppression, garde de session, échappement des sorties, test, commit.

- **M1 et M2** : recensement des ressources, puis wireframes ordinateur, tablette et mobile.
- **M3** : cahier des charges techniques (noté 20 sur 20) et dépôt GitHub public.
- **M4** : fiche descriptive E6.
- **M5** : base de données. **M6** : back-office. **M7** : tests. **M8** : GitHub et mise en ligne.

Chaque mission a donné un ou deux commits sur GitHub, dont le message commence par le numéro de la mission. L'historique du dépôt raconte donc l'avancement du projet.

3. Correction du code fourni

Avant de développer, j'ai relu tout le code de départ. J'ai trouvé 7 défauts, dont 2 qui empêchaient le site de fonctionner :

- Le filtre `FILTER_SANITIZE_STRING` est obsolète depuis PHP 8.1. Il affichait un avertissement, et la redirection après connexion échouait : on ne pouvait pas se connecter.
- La page du calendrier écrivait les noms de tables en minuscules. Sous Windows ça marche, mais sur un serveur Linux la table `epreuve` n'existe pas, car elle s'appelle `EPREUVE`.
- Un sport se supprimait par un simple lien, sans protection contre le CSRF. Les erreurs SQL s'affichaient au visiteur. Le texte était échappé deux fois, et le menu d'administration avait des liens cassés.

J'ai tout corrigé dans un commit dédié, avant d'écrire la moindre nouvelle fonctionnalité.

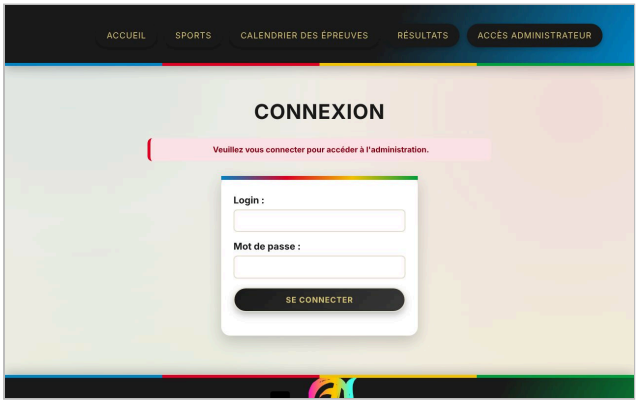
4. Mission 5 : la base de données

La base contient 8 tables en UTF-8 complet (`utf8mb4`) avec le moteur InnoDB : SPORT, LIEU, PAYS, GENRE, EPREUVE, ATHLETE, ADMINISTRATEUR et PARTICIPER. PARTICIPER est la table d'association entre les athlètes et les épreuves : sa clé primaire est composée des deux clés étrangères, donc un athlète a un seul résultat par épreuve.

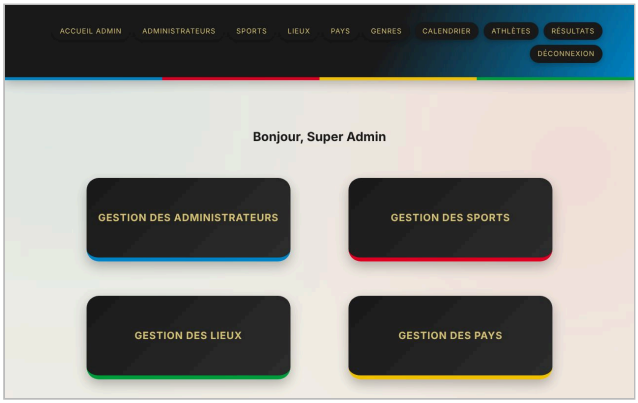
J'ai écrit un script de migration pour deux corrections. D'abord, la colonne `nom_epreuve` était prévue au modèle mais absente du script fourni : je l'ai ajoutée et remplie pour les épreuves existantes. Ensuite, j'ai ajouté 7 contraintes `UNIQUE` (nom du sport, du pays, du lieu, login de l'administrateur, athlète identifié par son nom, son prénom et son pays, un seul créneau par lieu). Les doublons sont ainsi contrôlés deux fois : en PHP pour afficher un message clair, et dans la base comme filet de sécurité.

5. Mission 6 : le back-office

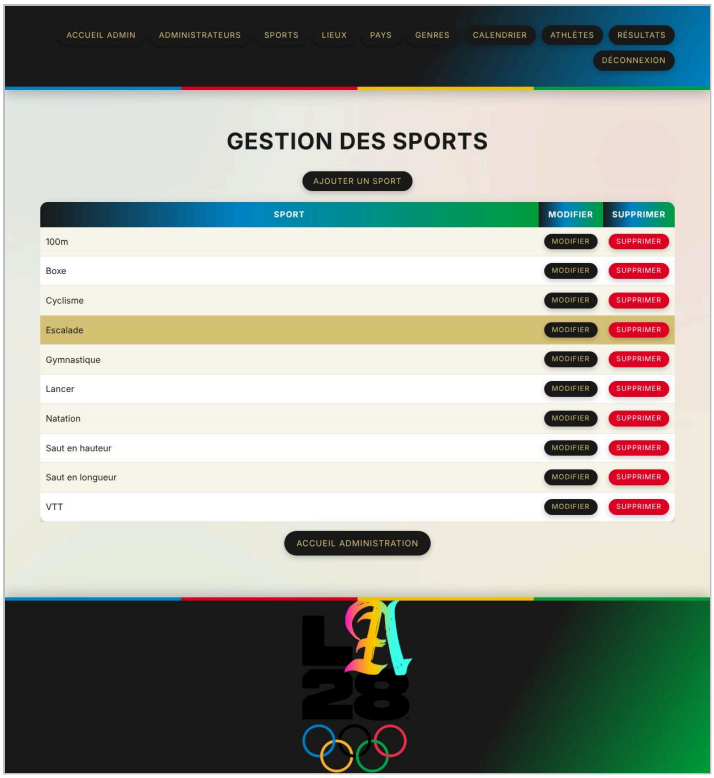
Le back-office compte 8 rubriques : administrateurs, sports, lieux, pays, genres, calendrier des épreuves, athlètes et résultats. Chacune a quatre pages : la liste, l'ajout, la modification et la suppression. Elles suivent toutes le même enchaînement : vérification que l'administrateur est connecté, vérification du jeton CSRF, lecture et contrôle des champs, contrôle des doublons, requête SQL préparée, message de confirmation, puis retour à la liste.



Page admin ouverte sans connexion : retour à la page de connexion



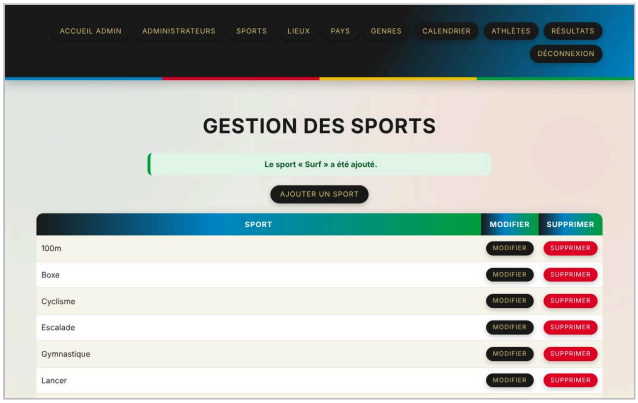
Accueil du back-office après connexion



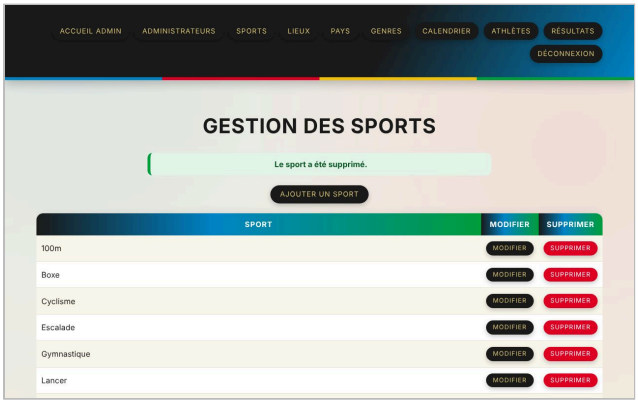
Gestion des sports : boutons Modifier et Supprimer sur chaque ligne



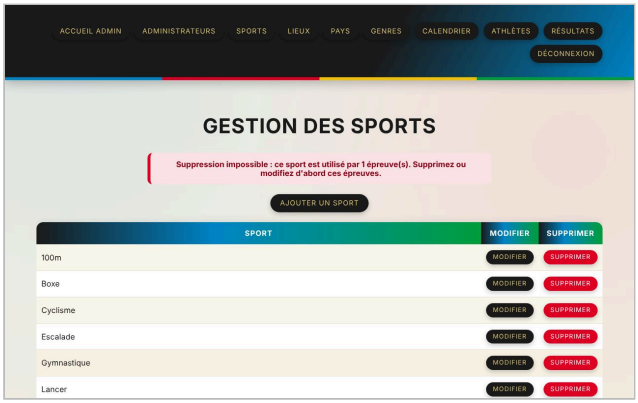
« natation » en minuscules est refusé : le sport existe déjà



Ajout d'un nouveau sport : message de confirmation



Suppression d'un sport, avec confirmation



Un sport utilisé par une épreuve ne peut pas être supprimé

Pour le calendrier, les athlètes et les résultats, les formulaires proposent des listes déroulantes remplies depuis la base (sport, lieu, pays, genre, athlète, épreuve). À la réception, je revérifie que l'identifiant

choisi existe bien en base, car un utilisateur peut modifier le formulaire dans son navigateur.



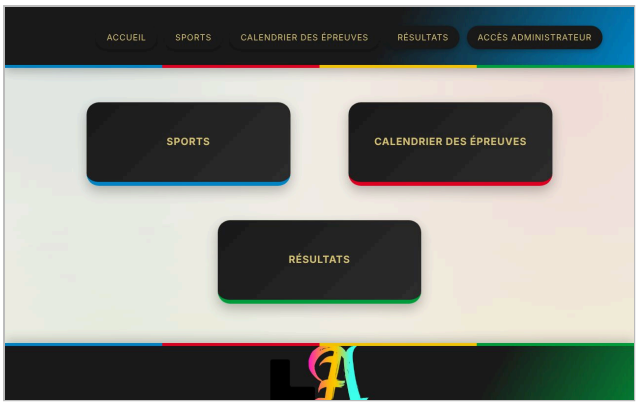
Ajout d'un résultat : listes déroulantes de l'épreuve et de l'athlète

6. La sécurité

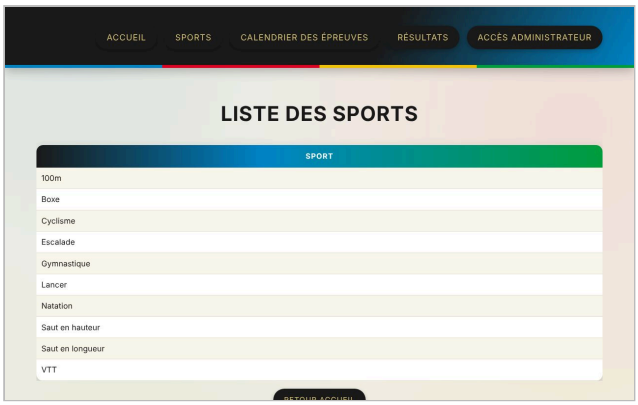
- **Injection SQL** : toutes les requêtes qui contiennent une saisie sont préparées avec PDO, et l'émulation des requêtes préparées est désactivée.
- **XSS** : chaque donnée affichée passe par `htmlspecialchars`.
- **CSRF** : chaque formulaire porte un jeton aléatoire de 64 caractères, comparé avec `hash_equals`. Les suppressions passent uniquement en POST.
- **Mots de passe** : empreinte Bcrypt avec `password_hash`, vérifiée avec `password_verify`. 8 caractères minimum et 72 octets maximum, la limite de Bcrypt.
- **Sessions** : nouvel identifiant à la connexion, cookies HttpOnly et SameSite Strict, déconnexion après 30 minutes d'inactivité.
- **Erreurs** : en production, le visiteur voit un message générique, le détail va dans le journal du serveur.

7. La partie publique

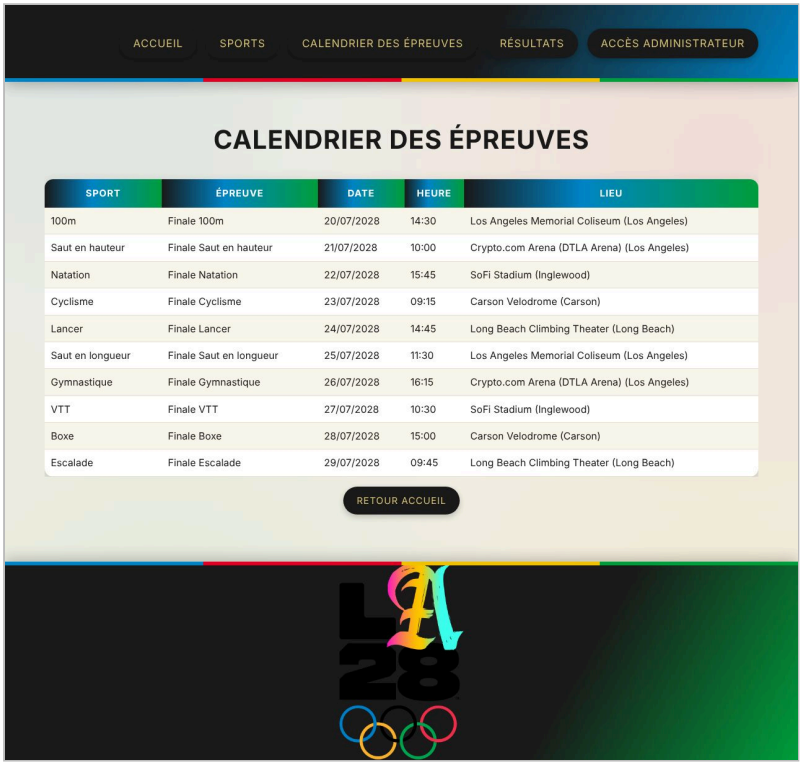
Les pages publiques assemblent les données de plusieurs tables avec des jointures SQL, et les dates sont mises au format français par MySQL. La mise en page s'adapte à l'ordinateur, à la tablette et au téléphone.



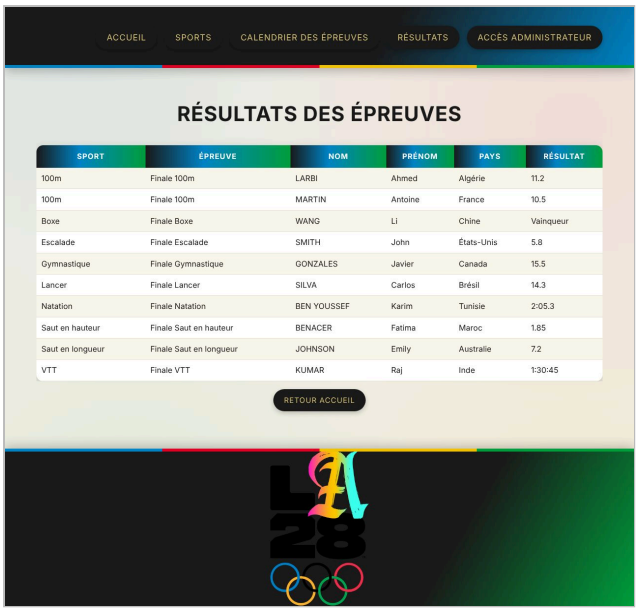
Page d'accueil



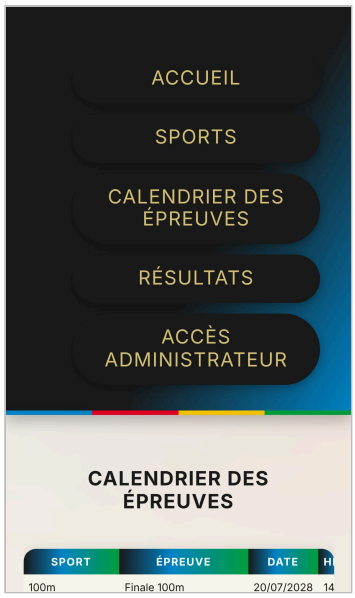
Liste des sports



Calendrier des épreuves : jointure EPREUVE, SPORT et LIEU



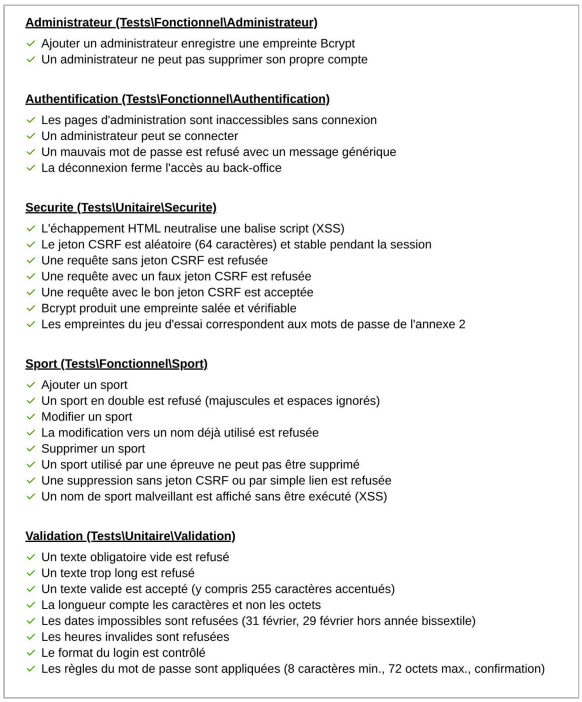
Résultats : 4 jointures, accents corrects (Algérie)



Le calendrier sur téléphone

8. Mission 7 : les tests

J'ai installé PHPUnit 11 avec Composer et écrit 29 tests, soit 77 vérifications. 15 tests unitaires contrôlent les fonctions de validation et de sécurité. 14 tests fonctionnels pilotent le vrai site, avec un serveur PHP de test et un navigateur simulé, sur une base de test recréée avant chaque test. Chaque test vérifie le message affiché et l'état réel de la base, y compris face aux attaques XSS et CSRF. Les 29 tests passent.



Administrateur (Tests\Fonctionnel\Administrateur)
- ✓ Ajouter un administrateur enregistre une empreinte Bcrypt
- ✓ Un administrateur ne peut pas supprimer son propre compte

Authentification (Tests\Fonctionnel\Authentification)
- ✓ Les pages d'administration sont inaccessibles sans connexion
- ✓ Un administrateur peut se connecter
- ✓ Un mauvais mot de passe est refusé avec un message générique
- ✓ La déconnexion ferme l'accès au back-office

Securite (Tests\Unitaire\Securite)
- ✓ L'échappement HTML neutralise une balise script (XSS)
- ✓ Le jeton CSRF est aléatoire (64 caractères) et stable pendant la session
- ✓ Une requête sans jeton CSRF est refusée
- ✓ Une requête avec un faux jeton CSRF est refusée
- ✓ Une requête avec le bon jeton CSRF est acceptée
- ✓ Bcrypt produit une empreinte salée et vérifiable
- ✓ Les empreintes du jeu d'essai correspondent aux mots de passe de l'annexe 2

Sport (Tests\Fonctionnel\Sport)
- ✓ Ajouter un sport
- ✓ Un sport en double est refusé (majuscules et espaces ignorés)
- ✓ Modifier un sport
- ✓ La modification vers un nom déjà utilisé est refusée
- ✓ Supprimer un sport
- ✓ Un sport utilisé par une épreuve ne peut pas être supprimé
- ✓ Une suppression sans jeton CSRF ou par simple lien est refusée
- ✓ Un nom de sport malveillant est affiché sans être exécuté (XSS)

Validation (Tests\Unitaire\Validation)
- ✓ Un texte obligatoire vide est refusé
- ✓ Un texte trop long est refusé
- ✓ Un texte valide est accepté (y compris 255 caractères accentués)
- ✓ La longueur compte les caractères et non les octets
- ✓ Les dates impossibles sont refusées (31 février, 29 février hors année bissextile)
- ✓ Les heures invalides sont refusées
- ✓ Le format du login est contrôlé
- ✓ Les règles du mot de passe sont appliquées (8 caractères min., 72 octets max., confirmation)

Rapport PHPUnit : 29 tests réussis

9. Mission 8 : la mise en ligne

Le code est sur GitHub sans aucun secret : le fichier de configuration avec le mot de passe de la base est exclu par `.gitignore`, et je publie seulement un modèle. Pour l'hébergement, j'ai utilisé InfinityFree. J'ai importé la base avec un script SQL de production qui impose le moteur InnoDB, car l'hébergeur créait les tables en MyISAM, un moteur qui ignore les clés étrangères. Un fichier `.htaccess` bloque l'accès aux scripts SQL et aux dossiers internes, et ajoute des en-têtes de sécurité.

J'ai envoyé les fichiers avec un script PowerShell. La première version passait par le composant FTP de .NET : la connexion chiffrée s'ouvrait, mais le premier fichier échouait avec l'erreur 451. Le serveur d'InfinityFree (Pure-FTPd) refusait le transfert, parce que ce composant ferme mal le canal de données chiffré. J'ai remplacé l'envoi par curl, l'outil de transfert fourni avec Windows, et les 60 fichiers sont passés du premier coup.

J'ai ensuite fait la recette en ligne : les pages publiques s'affichent avec les accents, les fichiers sensibles répondent « accès refusé », les dossiers internes répondent « page introuvable », et les pages d'administration renvoient vers la connexion quand on n'est pas connecté.

10. Bilan

Le site est terminé, testé et en ligne. Ce projet m'a appris à reprendre un code existant avant de le faire évoluer, à appliquer les protections de l'OWASP dans un vrai site, à écrire des tests automatisés et à mettre en production sur un hébergeur. La difficulté la plus formatrice a été la mise en ligne : lire le message d'erreur, trouver la cause, essayer une autre solution, puis vérifier le résultat.